Game

Items

- Objects that you can interact with, use on yourself, use on others
- Entities can carry them
- Some uses:
  - Attacking: Weapons, spells
  - Healing: Potions, hearts
  - Bonuses: Collecting items

Items

Engine provides base class:

```
// engine/item.h
class Item {
public:
    // name corresponds to a sprite in items.txt
    explicit Item(std::string name);

    // override what happens when the weapon is used in game
    // 1) use the item on yourself
    virtual void use(Engine& engine, Entity& owner);
    // 2) use the item on someone else
    virtual void use(Engine& engine, Entity& attacker, Entity& defender);

    // override if you want something to happen when an entity interacts
    // with this item, e.g. touching it, picking it up
    virtual void interact(Engine& engine, Entity& entity);
};
```

Creating Items

```
#include "item.h"

class Sword : public Item {
public:
    Sword(int damage);

    // cause damage to defender
    void use(Engine& engine, Entity& attacker, Entity& defender) override;
};
```

Creating Items

```
class Potion : public Item {
public:
    Potion(int healing_amount);

    // add health to owner
    void use(Engine& engine, Entity& owner) override;
};
```

Item locations

Items can be placed on Tiles

```
class Tile {
public:
    // simplified
    bool has_item() const;
    std::shared_ptr<Item> item;
};
```

Item locations

Items can be carried by Entities

```
class Entity {
public:
    // simplified
    // managing items within the inventory
    bool is_inventory_full() const;
    void add_to_inventory(std::shared_ptr<Item> item);
    std::shared_ptr<Item> remove_item(int index);

    // returns selected item number and names of all items in inventory
    std::pair<int, std::vector<std::string>> get_inventory_list() const;

    std::shared_ptr<Item> get_current_item() const;
    void select_item(int index);
};
```

Attacking

Big picture

- An Entity can use an **Item** to cause damage to another Entity
- The amount of damage is dependent on the **Item**
- Attacking is an **Action**, consequences are **Hit Events**
- **Hit Events** reduce health

Flow of an Attack

1. Entity generates **Attack Action** on Defender
2. Attack Action gets **current Item** from attacker
3. `Item::use(Engine& engine, Entity& attacker, Entity& defender)`
4. Weapon generates `Hit` Event with its damage amount
5. `Hit` Event calls `defender.take_damage(amount)`

Why all this indirection?

- Flexibility and separations of concerns
- Item properties determine damage amount
- Items can do anything to Attacker and Defender
  - Cursed Weapon damages attacker
  - Ultra powerful weapons damage both
  - Teleporation, confusion, game-ending

Why all this indirection?

- Animations
  - A simple scheme doesn't allow for flashy graphics

```
Entity::attack(Entity& defender, int amount) {
    // where's the animation?
    defender.take_damage(amount);
}
```

Events are used to order consequences (graphics, sound, damage, dying, etc)

Step 1: Attack Action

Entity generates **Attack Action** on Defender

```
// content/actions/attack.h
class Attack : public Action {
public:
    Attack(Entity& defender);
    // use current item on defender
    Result perform(Engine& engine, std::shared_ptr<Entity> attacker);

private:
    Entity& defender;
};
```

Update Move Action

If you move onto a tile that has entity, attack if it's an enemy!

```
// content/actions/move.cpp
Result Move::perform(Engine& engine, std::shared_ptr<Entity> entity) {
    ...
    if (tile has entity) {
        // if different team, then return Attack action
        // otherwise, Rest (since we are surrounded by friends)
    }
    ...
}
```

Step 2: Get current item

```
class Entity {
public:
    void take_damage(int amount);
    std::shared_ptr<Item> get_current_item() const;

private:
    // health gets reduced by calling take damage
    int health{1}, max_health{1};
    bool alive{true};

    // teams can be used to determine who can attack whom
    Team team;
};
```

Step 3: Use the item

```
class Item {
public:
    // use the item on someone else
    virtual void use(Engine& engine, Entity& attacker, Entity& defender);
};
```

Step 3: Use the item

```
class Sword : public Item {
public:
    Sword(int damage);

    // cause damage to defender
    void use(Engine& engine, Entity& attacker, Entity& defender) override;
};
```

`use()` will generate a **Hit** Event on defender

Step 4: Hit Events

- Events are very generic, override `execute()`
- Can be sequenced using `add_next()`

```
class Event {
public:
    explicit Event(int number_of_frames=1); // how many frames it should last

    // what the event does per update frame
    virtual void execute(Engine& engine) = 0;

    // called when the event is completed, useful for cleaning up
    // (e.g. resetting a sprite after an animation)
    virtual void when_done(Engine& engine);

    // add events that will take place after this current one
    template <typename T, typename... Args>
    void add_next(Args&&... args) {
        auto event = std::make_shared<T>(std::forward<Args>(args)...);
        next_events.push_back(event);
    }
    void add_next(std::shared_ptr<Event> event) {
        next_events.push_back(event);
    }

    // holds the events that will be run when this event is done
```

Step 4: Hit Events

```
#include "event.h"

// forward declaration
class Entity;

class Hit : public Event {
public:
    Hit(Entity& entity, int damage);
    void execute(Engine& engine) override;
    void when_done(Engine& engine) override;

private:
    Entity& entity;
    int damage;
};
```

Step 5: reducing health

Executing a Hit calls `Entity::take_damage()`

```
#include "hit.h"

#include "die.h"
#include "entity.h"

Hit::Hit(Entity& entity, int damage)
    :entity{entity}, damage{damage} {}

void Hit::execute(Engine&) {
    entity.take_damage(damage); // reduce health
}

void Hit::when_done(Engine&) {
    if (!entity.is_alive()) {
        add_next<Die>(entity); // remove from game
    }
}
```

Step 5: reducing health

Reduce health of entity

```
void Entity::take_damage(int amount) {
    health -= amount;
    health = std::clamp(health, 0, max_health);
    if (health == 0) {
        alive = false;
    }
}
```

Note: Healing can be implemented using `take_damage(-amount)`

Step 5: reducing health

Removing dead entities from the game

```
#include "event.h"

// forward declaration
class Entity;

class Die : public Event {
public:
    explicit Die(Entity& entity);
    void execute(Engine& engine) override;

private:
    Entity& entity;
};
```

Step 5: reducing health

```
#include "die.h"

#include "engine.h"
#include "entity.h"

Die::Die(Entity& entity)
    :entity{entity} {}

void Die::execute(Engine& engine) {
    engine.remove_entity(entity);
}
```

Demo: Creating a Weapon

- Go to `assets/items.txt` and choose a weapon sprite
- Choose a damage amount (choose value based on health values)
- Create a class for a weapon
    - Ideas: Sword, Axe, Hammer, Staff

Demo: Creating a Weapon

Let's make a Spear

```
// content/items/spear.h
#include "item.h"

class Spear : public Item {
public:
    explicit Spear(int damage);
    void use(Engine& engine, Entity& attacker, Entity& defender) override;

private:
    int damage;
};
```

```
// content/items/spear.cpp
Spear::Spear(int damage)
    :Item{"spear", damage} {}

void Spear::use(Engine& engine, Entity&, Entity& defender) {
    engine.events.create_event<Hit>(defender, damage);
}
```

Equipping items

```
// content/entities/heroes.cpp
void make_wizard(std::shared_ptr<Entity>& hero) {
    hero->set_sprite("wizard");
    hero->set_max_health(10);

    // add items to inventory
    hero->add_to_inventory(std::make_shared<Staff>(2));
    hero->add_to_inventory(std::make_shared<Spear>(4));

    hero->behavior = behavior;
}
```

Switching items

There are two methods: select item number (1-5) or cycle through items

```
std::unique_ptr<Action> behavior(Engine& engine, Entity& entity)
    std::string key = engine.input.get_last_keypress();
    ...
    else if (!key.empty() && std::isdigit(key.at(0))){
        int item_num = std::stoi(key);
        entity.select_item(item_num);
    }
    else if (key == "R") {
        entity.switch_to_next_item();
    }

    return nullptr;
}
```

Using Items

Let's make an action for using an Item, e.g. Drinking a potion

Checkpoint 3

- Add **Attack** action
- Modify **Move::perform()** to attack another entity if on a different team
- Add code for **Hit** and **Die** classes
- **Make two new weapons**, give them to hero/ monsters
- Make sure attacking works!